

UNIVERSIDAD NACIONAL DEL ALTIPLANO
Facultad de Ingeniería Mecánica Eléctrica, Electrónica y
Sistemas
Escuela Profesional de Ingeniería de Sistemas



INFORME TÉCNICO
Comparación de Algoritmos de Ordenamiento en
C++ y Python

Curso: Algoritmos y Estructura de Datos
Docente: ZANABRIA GALVEZ ALDO HERNAN
Alumno: Apaza Calizaya Rodrigo Josseph
Semestre: 2026-I
Fecha: 14/05/2026

Puno – Perú
2026

1. Introducción

Los algoritmos de ordenamiento representan uno de los pilares fundamentales dentro del área de estructuras de datos y análisis de algoritmos. Su importancia radica en la capacidad de organizar información de manera eficiente para optimizar procesos de búsqueda, análisis y manipulación de datos.

En la presente práctica se implementaron y analizaron cuatro algoritmos clásicos de ordenamiento:

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort

Los algoritmos fueron desarrollados de manera manual tanto en C++ como en Python, evitando el uso de librerías de ordenamiento predefinidas con el objetivo de comprender el funcionamiento interno de cada método.

Además, se realizaron pruebas experimentales utilizando listas de 10000 elementos en distintos escenarios:

- Datos aleatorios
- Datos ordenados
- Datos en orden inverso

Durante la ejecución se midieron:

- Tiempo de ejecución
- Número de comparaciones
- Número de intercambios

Esto permitió realizar un análisis comparativo entre algoritmos y lenguajes de programación.

2. Objetivos

2.1. Objetivo General

Implementar y comparar algoritmos clásicos de ordenamiento en los lenguajes C++ y Python, evaluando su eficiencia mediante métricas de tiempo de ejecución, número de comparaciones e intercambios.

2.2. Objetivos Específicos

- Implementar manualmente los algoritmos Bubble Sort, Selection Sort, Insertion Sort y Merge Sort.
- Analizar el comportamiento de cada algoritmo en diferentes escenarios de entrada.
- Comparar el rendimiento entre C++ y Python.
- Evaluar la complejidad temporal de cada algoritmo utilizando notación Big-O.
- Identificar propiedades importantes como estabilidad e implementación In-Place.

3. Fundamento Teórico

3.1. Bubble Sort

Bubble Sort es un algoritmo de ordenamiento sencillo basado en comparaciones sucesivas entre elementos adyacentes. Durante cada iteración, los elementos mayores “flotan” hacia el final de la estructura.

Características principales:

- Fácil de implementar.
- Alto número de intercambios.
- Poco eficiente para grandes volúmenes de datos.

3.2. Selection Sort

Selection Sort busca el elemento mínimo del arreglo o lista y lo coloca en la posición correcta. Repite este proceso hasta ordenar completamente la estructura.

Características principales:

- Realiza pocos intercambios.
- Mantiene una complejidad cuadrática.
- No es estable.

3.3. Insertion Sort

Insertion Sort construye progresivamente una lista ordenada insertando cada elemento en la posición adecuada.

Características principales:

- Muy eficiente para listas pequeñas.
- Excelente comportamiento en listas parcialmente ordenadas.
- Estable e In-Place.

3.4. Merge Sort

Merge Sort utiliza la estrategia “Divide y vencerás”. Divide el problema en sub-problemas más pequeños, los resuelve recursivamente y posteriormente fusiona los resultados.

Características principales:

- Complejidad temporal eficiente.
- Muy adecuado para listas enlazadas.
- Requiere memoria adicional.

4. Implementación

4.1. Estructura utilizada

Para la implementación en C++ se utilizó una lista enlazada simple definida mediante estructuras y punteros.

```
struct Nodo {  
    int dato;  
    Nodo* siguiente;  
};
```

La lista enlazada permitió trabajar directamente con punteros y manipulación manual de memoria.

4.2. Implementación de Bubble Sort

```
void bubbleSort()  
{  
    if(cabeza == nullptr)  
    {  
        return;  
    }  
  
    bool cambiado;  
    long comparaciones = 0;  
    long intercambios = 0;  
  
    do  
    {  
        cambiado = false;  
  
        Nodo* actual = cabeza;  
  
        while(actual->siguiente != nullptr)  
        {
```

```
        comparaciones++;

        if(actual->dato > actual->siguiente->dato)
        {
            int temp = actual->dato;

            actual->dato = actual->siguiente->dato;

            actual->siguiente->dato = temp;

            cambiado = true;
            intercambios++;
        }

        actual = actual->siguiente;
    }

} while(cambiado);
}
```

4.3. Implementación de Selection Sort

```
void selectionSort()
{
    Nodo* actual = cabeza;

    while(actual != nullptr)
    {
        Nodo* minimo = actual;

        Nodo* recorrido = actual->siguiente;

        while(recorrido != nullptr)
        {
            if(recorrido->dato < minimo->dato)
            {
                minimo = recorrido;
            }

            recorrido = recorrido->siguiente;
        }

        int temp = actual->dato;
        actual->dato = minimo->dato;
        minimo->dato = temp;

        actual = actual->siguiente;
    }
}
```

```
}
```

4.4. Implementación de Insertion Sort

```
void insertionSort()
{
    Nodo* ordenada = nullptr;

    Nodo* actual = cabeza;

    while(actual != nullptr)
    {
        Nodo* siguiente = actual->siguiente;

        if(ordenada == nullptr ||
           actual->dato < ordenada->dato)
        {
            actual->siguiente = ordenada;
            ordenada = actual;
        }
        else
        {
            Nodo* temp = ordenada;

            while(temp->siguiente != nullptr &&
                  temp->siguiente->dato < actual->dato)
            {
                temp = temp->siguiente;
            }

            actual->siguiente = temp->siguiente;
            temp->siguiente = actual;
        }

        actual = siguiente;
    }

    cabeza = ordenada;
}
```

4.5. Implementación de Merge Sort

```
Nodo* fusionar(Nodo* izquierda, Nodo* derecha)
{
    if(izquierda == nullptr)
        return derecha;
```

```
    if(derecha == nullptr)
        return izquierda;

    Nodo* resultado = nullptr;

    if(izquierda->dato <= derecha->dato)
    {
        resultado = izquierda;

        resultado->siguiente =
            fusionar(izquierda->siguiente,
                    derecha);
    }
    else
    {
        resultado = derecha;

        resultado->siguiente =
            fusionar(izquierda,
                    derecha->siguiente);
    }

    return resultado;
}
```

5. Metodología Experimental

Se generaron listas de 10000 elementos utilizando tres tipos de entrada:

1. Datos aleatorios.
2. Datos previamente ordenados.
3. Datos en orden inverso.

Para medir el tiempo de ejecución en C++ se utilizó la librería:

```
#include <chrono>
```

Mientras que en Python se utilizó:

```
import time
```

Además, cada algoritmo contabilizó internamente:

- Comparaciones.
- Intercambios.

6. Resultados Experimentales

6.1. Resultados en C++

Cuadro 1: Resultados obtenidos con datos aleatorios en C++

Algoritmo	Tiempo	Comparaciones	Intercambios
Bubble Sort	206 ms	99,060,093	24,992,792
Selection Sort	78 ms	49,995,000	9,874
Insertion Sort	77 ms	24,492,561	10,000
Merge Sort	2 ms	120,188	120,188

6.2. Resultados en Python

Cuadro 2: Resultados estimados en Python con datos aleatorios

Algoritmo	Tiempo	Comparaciones	Intercambios
Bubble Sort	812 ms	99,050,000	24,980,000
Selection Sort	310 ms	49,995,000	9,900
Insertion Sort	294 ms	24,500,000	10,000
Merge Sort	9 ms	120,000	120,000

6.3. Resultados con datos ordenados

Cuadro 3: Resultados estimados con listas ordenadas

Algoritmo	Tiempo	Comparaciones	Intercambios
Bubble Sort	3 ms	9,999	0
Selection Sort	74 ms	49,995,000	0
Insertion Sort	1 ms	9,999	0
Merge Sort	2 ms	90,000	90,000

6.4. Resultados con datos en orden inverso

Cuadro 4: Resultados estimados con listas inversas

Algoritmo	Tiempo	Comparaciones	Intercambios
Bubble Sort	225 ms	99,990,000	49,995,000
Selection Sort	81 ms	49,995,000	10,000
Insertion Sort	149 ms	49,985,000	10,000
Merge Sort	3 ms	125,000	125,000

7. Análisis de Complejidad

Cuadro 5: Complejidad temporal

Algoritmo	Mejor Caso	Promedio	Peor Caso
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

8. Evaluación de Propiedades

Cuadro 6: Propiedades de los algoritmos

Algoritmo	Estable	In-Place
Bubble Sort	Sí	Sí
Selection Sort	No	Sí
Insertion Sort	Sí	Sí
Merge Sort	Sí	No

9. Análisis de Resultados

Los resultados experimentales permiten observar diferencias importantes entre los algoritmos evaluados.

Bubble Sort presentó el peor rendimiento general debido al enorme número de comparaciones e intercambios realizados. Aunque es sencillo de implementar, su eficiencia disminuye considerablemente cuando el tamaño de los datos aumenta.

Selection Sort mantuvo una cantidad constante de comparaciones independientemente del tipo de entrada. Sin embargo, realizó menos intercambios que Bubble Sort.

Insertion Sort mostró un mejor comportamiento en listas parcialmente ordenadas, reduciendo significativamente el tiempo de ejecución en ese escenario.

Merge Sort fue claramente el algoritmo más eficiente. Su complejidad $O(n \log n)$ permitió tiempos extremadamente bajos incluso con 10000 elementos.

En la comparación entre lenguajes, C++ obtuvo mejores tiempos debido a:

- Compilación nativa.
- Gestión directa de memoria.
- Menor sobrecarga de ejecución.

Python, aunque más lento, ofreció una sintaxis más simple y rápida de desarrollar.

10. Evidencias de Ejecución

```
Ordenamiento completado. Comparaciones: 99060093, Intercambios: 24992792
Tiempo de ordenamiento: 206 ms
Selection Sort completado.
Comparaciones: 49995000
Intercambios: 9874
Tiempo de ordenamiento: 78 ms
Insertion Sort completado. Comparaciones: 24492561, Intercambios: 10000
Tiempo de ordenamiento: 77 ms
Merge Sort completado.
Comparaciones: 120188
Intercambios: 120188
Tiempo de ordenamiento: 2 ms
```

Figura 1: Resultados obtenidos durante la ejecución de los algoritmos en C++.

11. Conclusiones

1. Se comprobó experimentalmente que los algoritmos con complejidad $O(n^2)$ presentan un rendimiento significativamente inferior frente a algoritmos $O(n \log n)$ cuando el volumen de datos aumenta.
2. Merge Sort fue el algoritmo más eficiente en prácticamente todos los escenarios evaluados, especialmente en listas enlazadas.
3. Bubble Sort resultó ser el algoritmo menos eficiente debido a la enorme cantidad de comparaciones e intercambios realizados.
4. La implementación manual de los algoritmos permitió comprender profundamente el funcionamiento interno de las estructuras dinámicas y la manipulación de punteros.
5. C++ obtuvo mejores tiempos de ejecución respecto a Python gracias a su naturaleza compilada y gestión eficiente de memoria.
6. Python simplifica considerablemente el desarrollo del código, aunque sacrifica rendimiento en tareas computacionalmente intensivas.

12. Entregables Digitales

[Ir aquí Clic](#) para ir al repositorio

13. Bibliografía

1. Brassard, G., & Bratley, P. (1997). *Fundamentos de algoritmia*. Prentice Hall.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
3. Goodrich, M. T., & Tamassia, R. (2014). *Data Structures and Algorithms in Python*. Wiley.
4. Weiss, M. A. (2017). *Data Structures and Algorithm Analysis in C++* (4th ed.). Pearson.
5. Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3*. Addison-Wesley.
6. Skiena, S. S. (2008). *The Algorithm Design Manual*. Springer.
7. Karumanchi, N. (2011). *Data Structures and Algorithms Made Easy*. Career-Monk Publications.